Final Project Design Phase

Computer Science - Week 11

Oct 3, 2026 - Version 0.0.4

For complete instructions and project ideas, refer to the assignment in Canvas. Specifically, review the final project requirements before you continue below. If you have not reviewed the project requirements in Canvas (you might have to go to the modules view and check the current or past week's content), you will not be able to properly complete the parts below. In addition, it may be helpful to review the information on this Drafter documentation page.

0) Project Planning and Preparation

Successful programmers spend time planning before they begin coding. *After* reviewing the project requirements, answer the following questions to help guide your design and development process.

A. What concepts from class or previous assignments might help you complete this project? Which programming concepts (loops, conditionals, lists, classes, etc.) do you expect to use? What programming skills do you want to improve while completing this project?

I will definitely need lots of loops, classes, and lists. I would like to get better at loops and slightly better at dataclasses because I've always struggled with them but they're a very heavy part of my code for this project.

B. Which parts of the project do you expect to be the easiest for you? The hardest? Why?

I think the actually mathematics/arithmetic inside of the coding will be the easiest part because I know exactly what calculations need to be done and how to convert them into code.

C. Even though you have not yet designed the final project, estimate which tasks will likely require the most time and effort? Which features or components should be implemented first?

I think making all of the different scenarios into separate pages on Drafter is definitely

gonna take a good bulk of my time. I will implement all the small codes first, basically starting in and expanding out.

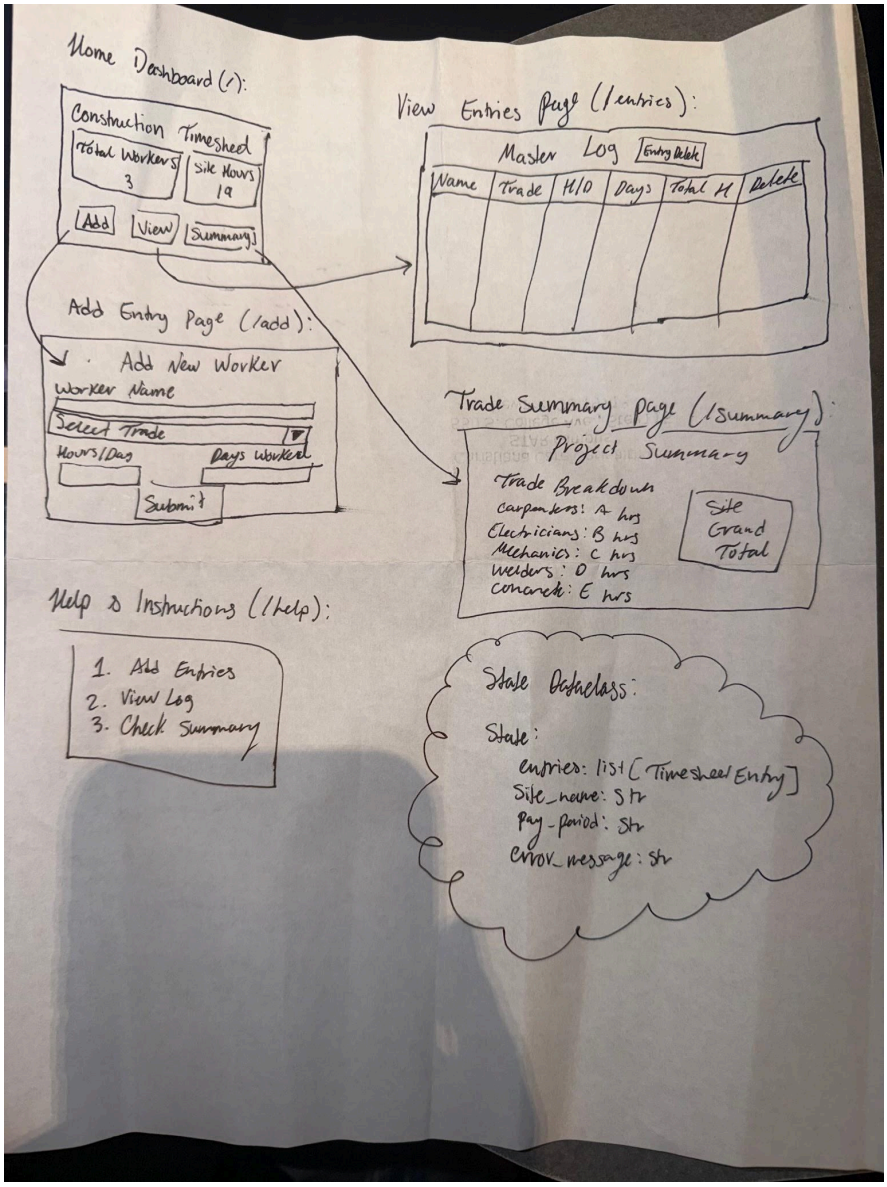
D. What distractions might interfere with your work and how will you minimize them? What resources will you need available while working? What questions, if any, do you still have about the assignment requirements?

Other course work and responsibilities will definitely interfere with my progression on this project. I will try to minimize them by setting a designated day and time to make progress on this project so that other commitments can't interfere. As of right now I don't need any resources but I'm sure I will be at office hours pretty soon. I don't have any outstanding questions at the moment.

1) Site Diagram

Create a mock-up of your planned website. We suggest you make the mockup on paper, although you can also use software like PowerPoint or Slides to make a mockup. There is other design software that could also be used, but we strongly encourage you to keep it simple. Do not spend more than 15-30 minutes detailing the mock-up. Your goal should be to make a very quick and dirty diagram, to show that you have some kind of idea.

We recommend drawing each page as a rectangle, and laying out the text, buttons, and other components inside. Then, use arrows to indicate links between pages. Label each page the way you will name each function, and then put labels on the arrows to indicate what the button links. Also write down any parameters that should come into the pages.



In addition to the pages and routes, write down what fields your `State` will have, both their names and types. Think clearly about what data will need to be persistent across the entire website.

entries (list[TimesheetEntry]): A persistent list storing all recorded worker time entries for the active session.

site_name (str): Stores the name of the active construction project or job site.

pay_period (str): Specifies the current timesheet billing or tracking cycle.

error_message (str): Holds feedback and validation warning messages when user inputs require correction.

Add photo(s) or screenshot(s) of your website mock-up and labeled diagram on the next page.

2) Implementing Requirements

This table should serve as a draft for how you will implement the requirements into your final project. You may change your ideas later on, but your final site must fulfill all the requirements.

Requirement	**Implementation**
Purpose/Functionality A 1-2 sentence description of the purpose or functionality of your site	This website allows construction project managers to manually log workforce hours by worker name and trade, automatically calculate individual and aggregate labor hours, and review organized summaries by worker and trade to simplify timesheet record-keeping.
State Dataclass Attributes Minimum of 4 attributes, one of which must be a list/dict of instances of your second dataclass. Briefly describe each attribute, its type, and how it will be meaningfully modified throughout the site.	• entries (list[TimesheetEntry]): A list holding instances of the TimesheetEntry dataclass. It is modified when users submit new entries or delete incorrect ones • site_name(str): Stores the name of the construction project. Modified when the user initializes or updates the site details • pay_period(str): Represents the active timesheet period. Modified when setting up the tracking cycle. • error_message(str): Holds feedback strings. Modified whenever input validation fails or passes successfully.
Second Dataclass Your second dataclass, including its attributes, their types, and how they will be meaningfully modified (if they are modified)	**Attributes & Types**: worker_name (str): The name of the worker. trade (str): The construction trade (e.g., Carpenter, Electrician, Laborer, Operator). hours_per_day (float): Average hours worked per day. days_worked (int): Number of days worked in the period. total_hours (float): Calculated total gross hours

	(hours_per_day * days_worked). **Modification**: Created upon valid form submission and appended to the State's entries list; individual entries can be removed via deletion handlers.
Routes (minimum 7) The names of your route functions	home_route (/) add_entry_route (/add) submit_entry_route (/submit) view_entries_route (/entries) trade_summary_route (/summary) delete_entry_route (/delete/{index}) help_route (/help)
Pages (minimum 5) A brief description of the pages in your site	1. **Home Dashboard**: Central hub displaying quick navigation and project overview statistics. 2. **Add Entry Page**: Form interface for inputting worker details, trade selection, and hours. 3. **View Entries Page**: Detailed table showing all recorded workforce entries with management options. 4. **Trade Summary Page**: Analytical breakdown aggregating total hours per construction trade and grand total site hours. 5. **Help & Instructions Page**: Guide explaining how to record data and interpret totals.
User Input (minimum 3) Description of user input components (TextBox, TextArea, CheckBox, or SelectBox) and their purposes For more information on these input types, see Drafter	TextBox for Worker Name: Captures the individual employee's name for record identification. SelectBox for Trade Selection: Restricts input to predefined construction trades (Carpenter, Electrician, Laborer, Operator) to maintain consistent data categorization. TextBox for Hours Worked per Day: Accepts numerical input representing daily shift hours. TextBox for Days Worked: Accepts numerical input representing total days worked during the period.

Documentation.	
If Statements (minimum 3) Brief description of what will be evaluated in your if statements and how they will meaningfully change the logic of the site	**Input Validation Check**: Evaluates if the worker name is blank or numerical values are negative/zero; if true, it triggers an error message and prevents entry creation. **Trade Filtering / Subtotal Logic**: Evaluates whether a specific entry's trade matches a target category during summary aggregation to compute accurate trade subtotals. **List Empty Check**: Evaluates if the entries list is empty before rendering summary tables, displaying a friendly placeholder message if no data exists yet.
Loop (minimum 1) This loop must iterate over the list/dict of dataclass instances in your State attribute. Description of this loop and why it is meaningful to the site	**Timesheet Aggregation Loop**: Iterates over the entries list in the State dataclass to calculate the grand total site hours and individual worker total hours (hours_per_day * days_worked), ensuring all submitted records are processed dynamically.

LLM Conversation

If you used an LLM in any part of this work, copy and paste every part of that conversation below. Otherwise, you may be given a zero for the assignment. Obviously, these words will not all be yours, and this is the exception to the requirement at the top of this assignment that all responses be in your own words. If you did not use an LLM to assist you in this assignment, simply say so below.

Complete Conversation & Project Documentation

Developer: Chasiti Bobet-Diaz (chasbd@udel.edu)

Project: Construction Site Timesheet Tracker

Course: CS1 Final Project

Part 1: Transcript of Conversation

User Prompt 1

Based on this project, fill out these questions...

2) Implementing Requirements

This table should serve as a draft for how you will implement the requirements into your final project. You may change your ideas later on, but your final site must fulfill all the requirements.

Requirement	**Implementation**
Purpose/Functionality	A 1-2 sentence description of the purpose or functionality of your site
State Dataclass Attributes	Minimum of 4 attributes, one of which must be a list/dict of instances of your second dataclass. Briefly describe each attribute, its type, and how it will be meaningfully modified throughout the site.
Second Dataclass	Your second dataclass, including its attributes, their types, and how they will be meaningfully modified (if they are modified)
Routes (minimum 7)	The names of your route functions
Pages (minimum 5)	A brief description of the pages in your site
User Input (minimum 3)	Description of user input components (TextBox, TextArea, CheckBox, or SelectBox) and their purposes

If Statements (minimum 3)	Brief description of what will be evaluated in your if statements and how they will meaningfully change the logic of the site
Loop (minimum 1)	This loop must iterate over the list/dict of dataclass instances in your State attribute. Description of this loop and why it is meaningful to the site

[Attached document: CBD - CS1 Final Project - Pick a Problem (1).docx]

Assistant Response 1

Here are the completed answers for your Construction Site Timesheet Tracker project, formatted and ready to be transferred into your project assignment document.

2) Implementing Requirements

Purpose/Functionality

This web application allows construction project managers and superintendents to manually log workforce hours by worker name and trade, automatically calculate individual and aggregate labor hours, and review organized summaries by worker and trade to simplify timesheet record-keeping.

State Dataclass Attributes

- entries (list[TimesheetEntry]): A list holding instances of the TimesheetEntry dataclass. It is modified when users submit new entries or delete incorrect ones.
- site_name (str): Stores the name of the construction project. Modified when the user initializes or updates the site details.
- pay_period (str): Represents the active timesheet period. Modified when setting up the tracking cycle.
- error_message (str): Holds feedback strings. Modified whenever input validation fails or passes successfully.

Second Dataclass (**TimesheetEntry**)

- **Attributes & Types**:
  - worker_name (str): The name of the worker.
  - trade (str): The construction trade (e.g., Carpenter, Electrician, Laborer, Operator).

    - hours_per_day (float): Average hours worked per day.
    - days_worked (int): Number of days worked in the period.
    - total_hours (float): Calculated total gross hours (hours_per_day * days_worked).
- **Modification**: Created upon valid form submission and appended to the State's entries list; individual entries can be removed via deletion handlers.

Routes (Minimum 7)

1. home_route (/)
2. add_entry_route (/add)
3. submit_entry_route (/submit)
4. view_entries_route (/entries)
5. trade_summary_route (/summary)
6. delete_entry_route (/delete/{index})
7. help_route (/help)

Pages (Minimum 5)

1. **Home Dashboard**: Central hub displaying quick navigation and project overview statistics.
2. **Add Entry Page**: Form interface for inputting worker details, trade selection, and hours.
3. **View Entries Page**: Detailed table showing all recorded workforce entries with management options.
4. **Trade Summary Page**: Analytical breakdown aggregating total hours per construction trade and grand total site hours.
5. **Help & Instructions Page**: Guide explaining how to record data and interpret totals.

User Input (Minimum 3)

1. TextBox for Worker Name: Captures the individual employee's name for record identification.
2. SelectBox for Trade Selection: Restricts input to predefined construction trades (Carpenter, Electrician, Laborer, Operator) to maintain consistent data categorization.
3. TextBox for Hours Worked per Day: Accepts numerical input representing daily shift hours.
4. TextBox for Days Worked: Accepts numerical input representing total days worked during the period.

If Statements (Minimum 3)

1. **Input Validation Check**: Evaluates if the worker name is blank or numerical values are negative/zero; if true, it triggers an error message and prevents entry creation.
2. **Trade Filtering / Subtotal Logic**: Evaluates whether a specific entry's trade matches a target category during summary aggregation to compute accurate trade subtotals.
3. **List Empty Check**: Evaluates if the entries list is empty before rendering summary tables, displaying a friendly placeholder message if no data exists yet.

Loop (Minimum 1)

- **Timesheet Aggregation Loop**: Iterates over the entries list in the State dataclass to calculate the grand total site hours and individual worker total hours (hours_per_day * days_worked), ensuring all submitted records are processed dynamically.

Final Submission

When you are finished, download this file as a Word Document (docx) and upload the file to the appropriate assignment on Canvas.